

Elective Surgery Scheduling

An Interval-Based Constraint Programming Model

Google OR-Tools CP-SAT (primary) · MILP comparison (CBC/Gurobi) · Hexaly (optional)

Operations Research Case Study

June 26, 2026

Roadmap

- 1 The Case
- 2 Why Constraint Programming, Not a Bigger MILP
- 3 Scope, Assumptions & What's Excluded
- 4 The Model
- 5 Results
- 6 Visual Schedule
- 7 Real-World Validation
- 8 Open Questions
- 9 Roadmap

The Case, in One Slide

A large hospital group needs to decide, for every elective surgical case on its waiting list:

- **Which procedure runs in which operating room, at what time**
- **On which day**, across a **one-week** planning horizon
- **With which surgeon**
- And, since not everything fits this week: **which cases wait**

What we were asked for (two deliverables):

- ① A problem formulation: sets, parameters, decision variables, objective, constraints, *and the assumptions behind them* – in actual math. We choose what to include and exclude, and say why.
- ② A working demo solver: small data, runnable Python, produces a schedule. Plain terminal output or a simple image is enough.

We were explicitly invited to simplify and to ground choices in evidence (papers, real data, industry pain points) – and explicitly told this should stay small: “a well-reasoned model with a small demo beats an exhaustive one.”

Why This Is Hard – and Why It Matters

The bottleneck is real, and documented:

- Portugal's SIGIC system: of ~7,400 waiting-list patients audited in 2016, **16% had already breached** their clinical maximum-wait deadline, by an average of **147 days** (Marques & Captivo, 2015).
- The UK NHS Referral-to-Treatment framework and Canadian provincial wait-time benchmarks track the same kind of breach rate, for the same reason.
- Cardoen, Demeulemeester & Beliën (2010): *the* literature review for this problem family – classifies it, doesn't solve one country's instance of it.

NP-hard, structurally:

- Room-capacity feasibility alone reduces to bin packing.
- Add tiered priority, surgeon limits, shared equipment, and downstream beds, and the *choice of solver paradigm* – not just the solver brand – is the central technical decision in this deck.

Evidence used to *justify the model's shape* (priority + waiting time + escalating penalty) – not as a literal country-specific subject. Every numeric value is an instance-level, overridable parameter in the code.

The Central Modeling Decision

A linear capacity-sum constraint (“total minutes used this day $\leq k$ ”) only certifies that a set of durations *fits* – not that they can be placed *without colliding*. For a single room those coincide. They stop coinciding the moment a resource is **shared across rooms** – a surgeon in two rooms, one imaging unit serving two rooms:

- A surgeon’s daily-minutes *sum* can be within limit while two of their cases, in two rooms, are scheduled to overlap – physically impossible, invisible to a sum.
- A shared device’s day-count cap can be *needlessly tighter* than reality: it forbids two genuinely non-overlapping uses a real device could serve back-to-back.
- A day-bucket model has **no correct way to express** a multi-day recovery-bed stay that starts on the day of surgery – a constraint the case prompt names directly.

RESULTS.md shows this is not theoretical: a comparison MILP run on the same data finds a schedule that is forbidden outright by its own day-bucket equipment constraint, even though a strictly better, legal one exists.

Why CP-SAT, Specifically

The textbook MILP alternative – a disjunctive big-M formulation, one binary “A-before-B” variable per potentially-conflicting case pair – works, but (Baptiste, Le Pape & Nuijten, *Constraint-Based Scheduling*, 2001): is quadratic in the number of conflicting pairs, and resource *sharing* (capacity > 1) needs combinatorially more pairs still.

NoOverlap / Cumulative are **global constraints with their own polynomial propagation**: NoOverlap via Vilím’s $O(n \log n)$ Theta-tree sweep (2004); Cumulative via timetabling + edge-finding (Schutt, Feydy, Stuckey & Wallace, CP 2009). They prune the search tree directly from temporal structure, instead of the solver *discovering* it case-pair by case-pair through branching.

CP-SAT’s engine: a parallel portfolio of complete-search workers plus Large Neighbourhood Search workers sharing learned clauses (Perron & Furnon, OR-Tools docs) – which is why we don’t hand-write a custom search strategy, and why we *do* warm-start it from a fast greedy heuristic’s assignment (`model.AddHint(...)`; discrete assignment only, never exact clock times).

Assumptions – Stated, Not Hidden

- 1 **Deterministic durations** – one estimate per case (e.g. historical median); standard tractable approximation (Denton et al., 2010); stochastic extension named, not attempted, to keep this demo small.
- 2 **Fixed turnover/cleaning buffer**, added to every case's room-occupation interval – not modeled as a separate sequence-dependent activity.
- 3 **Surgeons are the binding staffing resource**; nurses/anaesthetists assumed pre-rostered to the room.
- 4 **One occurrence per patient per week** – conservative default.
- 5 **Recovery/ICU beds modeled, with a constant-capacity caveat**: bed *capacity* is constant across the week; day-varying capacity would need a segmented version (flagged, not silently assumed away).
- 6 **One ad hoc institutional rule, as a worked example** – a configurable “pediatric block” carve-out, to show such rules cost nothing structurally.
- 7 **Single week, no same-day disruption**. An offline weekly plan – not a reactive re-scheduler for an emergency walking in mid-day. Named explicitly as the most valuable next extension (Roadmap), not folded in here.

What's In, What's Out – and Why

Included

- Priority + waiting-time penalty (evidence-based, multi-system)
- Exact room/surgeon timing (no double-booking, not just a budget)
- Surgeon daily/weekly limits
- Room-service roster, pediatric-block worked example
- Exact shared-equipment concurrency
- Downstream recovery/ICU beds (case prompt names this directly)

Excluded, named explicitly

- Stochastic durations – top extension, not attempted here
- Same-day / real-time re-optimization – a different problem
- Nurses/anaesthetists as separate resources
- Sequence-dependent (vs. constant) turnover time
- Patient day-of-week preference
- Multi-week rolling horizon

Sets and Parameters

Symbol	Description
$c \in C, d \in D = \{1, \dots, 5\}, r \in R, h \in H, e \in E$ $D_c \subseteq D$ $t_c^{\text{op}}, t_c^{\text{clean}}, t_c^{\text{tot}}$ k_{dr}, k_{hd}, k_h $a_{dr}^s \in \{0, 1\}$ $p_c \in \{1, 2, 3, 4\}, dd_c, \mu_p, \alpha, w_c$ $u_{ce} \in \{0, 1\}, \kappa_{ed}$ $\rho(c), \text{los}_c, \beta_\rho$	Cases, days, rooms, surgeons, shared equipment Days case c may run ($\{1\}$ if priority 4, else D) Operative / cleaning / total room time for case c Room capacity; surgeon daily / weekly time limit 1 if room r on day d is rostered to service s Priority; slack to deadline; penalty multiplier; urgency mult.; non-scheduling weight Equipment e required by case c; capacity of e on day d Bed pool needed; length of stay; bed count for pool ρ

Every default above is an instance-level, overridable field on `PlanningInstance` – not a hard-coded constant.

Decision Variables and Objective

For every (c, d, r) surviving the eligibility pre-filter (room-service roster, ambulatory-only, pediatric block, surgeon availability):

$$pr_{cdr} \in \{0, 1\}, \quad start_{cdr}, end_{cdr} \in [0, k_{dr}], \quad iv_{cdr} = \text{NewOptionalIntervalVar}(start_{cdr}, t_c^{\text{tot}}, end_{cdr}, pr_{cdr})$$

and, for every non-priority-4 case, $u_c \in \{0, 1\}$ with $\sum_{d,r} pr_{cdr} + u_c = 1$.

$$\begin{aligned} \min \quad & \underbrace{\sum_{c: dd_c \geq 0} \sum_{d,r} [dd_c + d] pr_{cdr}}_{\text{on-time, prefer earlier days}} + \underbrace{\sum_{c: dd_c < 0} \sum_{d,r} [dd_c + \alpha d] pr_{cdr}}_{\text{overdue, urgency mult. } \alpha} + \underbrace{\sum_{c: p_c \neq 4} p_c w_c u_c}_{\text{non-scheduling penalty}} \\ & w_c = \text{PenaltyCurve}(dd_c \cdot \mu_{p_c}) + 1.2 \cdot \max_{c' \in C} dd_{c'} \quad (\text{calibrated to dominate Terms 1-2}) \end{aligned}$$

Constraints I – Case Disposition & Eligibility

C1 – at most one occurrence per patient per week: $\sum_{c:\text{patient}(c)=n} \sum_{d,r} \text{pr}_{cdr} \leq 1 \ \forall n$. **C2** – priority-4 cases must run on day 1: $\sum_r \text{pr}_{c,1,r} = 1 \ \forall c : p_c = 4$. **C3** – every other case scheduled exactly once, or penalised: $\sum_{d,r} \text{pr}_{cdr} + u_c = 1$.

C4–C6 (pre-filter, not separate rows): room-service roster ($a_{dr}^{\text{service}(c)} = 0 \Rightarrow$ no candidate); ambulatory-only rooms admit only day-case scopes; pediatric block forbids age $> i^\dagger$ in service $s^\dagger$'s rooms on day $d^\dagger$.

These three eliminate infeasible (c, d, r) triples before the model is even built – the same effect as a constraint row, at a fraction of the variable count.

Constraints II – Capacity & Shared Resources

C7 – room capacity, exact: $\text{AddNoOverlap}(\{\text{iv}_{cdr} : d, r \text{ fixed}\})$. **C8 – surgeon, exact + daily cap:** $\text{AddNoOverlap}(\{\text{iv}_{cdr} : \text{surgeon}(c) = h, d \text{ fixed}\})$ plus $\sum_c t_c^{\text{op}} \text{pr}_{cdr} \leq k_{hd}$. **C9 – surgeon weekly limit:** $\sum_{d,r} t_c^{\text{op}} \text{pr}_{cdr} \leq k_h$.

C10 – shared equipment, exact concurrency:

$\text{AddCumulative}(\{\text{iv}_{cdr} : u_{c,e} = 1, d \text{ fixed}\}, \text{demands} = 1, \text{capacity} = \kappa_{ed})$

C11 – downstream recovery/ICU beds:

$\text{bed}_c = \text{NewOptionalIntervalVar}(\text{dayof}_c, \text{los}_c, \text{dayof}_c + \text{los}_c, \text{is_scheduled}_c), \quad \text{AddCumulative}(\{\text{bed}_c : \rho(c) = \rho\}, 1, \beta_\rho)$

C7–C11 share one interval per candidate slot – a case's start propagates across every resource it touches without an explicit channeling constraint.

Alternative Formulations Considered

A day-bucket MILP (`milp_baseline_solver.py`, OR-Tools/CBC, optionally Gurobi) was also built – same C1–C6, C9; C7/C10 as linear sums instead of NoOverlap/Cumulative; no C11 (can't express a multi-day bed stay correctly without intervals). It exists **purely as the empirical comparison that justifies the CP-SAT choice** – not a second deliverable. Results next section.

Hexaly (`hexaly_solver.py`) is a real, non-stub local-search integration, written against a set-partition encoding – an **optional, future contribution**: no academic license was available while building this, so it runs only as a graceful fallback to the MILP above today, with setup instructions printed at runtime. Pointed at as the natural backend for very large instances or real-time re-optimization, not benchmarked here.

Results – Demo Instance (20 cases, 5 rooms, 6 surgeons)

Solver	Status	Objective	Gap	Scheduled	Time
Greedy	Feasible	163.0	–	20/20	0.000s
CP-SAT (primary)	Optimal	155.0	0.00%	20/20	0.10s
OR-Tools/CBC (comparison MILP)	Optimal	157.0	0.00%	20/20	0.03s
Gurobi (comparison MILP)	Optimal	157.0	0.00%	20/20	0.06s
Hexaly (→ CBC fallback)	Optimal	157.0	0.00%	20/20	0.02s

CP-SAT wins on objective (155 vs. 157), not by searching harder, but by modeling the shared C-arm correctly: AddCumulative checks literal time overlap, so it legally places two C-arm cases on Tuesday, in different rooms, at non-overlapping times – a schedule the MILP's day-count cap forbids outright.

Results – Medium Instance (200 cases, 12 rooms, 17 surgeons)

60-second budget per solver.

Solver	Status	Objective	Gap	Scheduled	Time
Greedy	Feasible	70,883.0	–	124/200	0.00s
CP-SAT (primary)	Feasible	41,548.0	4.30%	130/200	60.45s
OR-Tools/CBC (comparison MILP)	Feasible	44,346.0	0.31%	128/200	60.11s
Gurobi (comparison MILP)	Feasible	44,333.0	0.45%	129/200	1.65s
Hexaly (→ CBC fallback)	Feasible	44,346.0	0.31%	128/200	60.13s

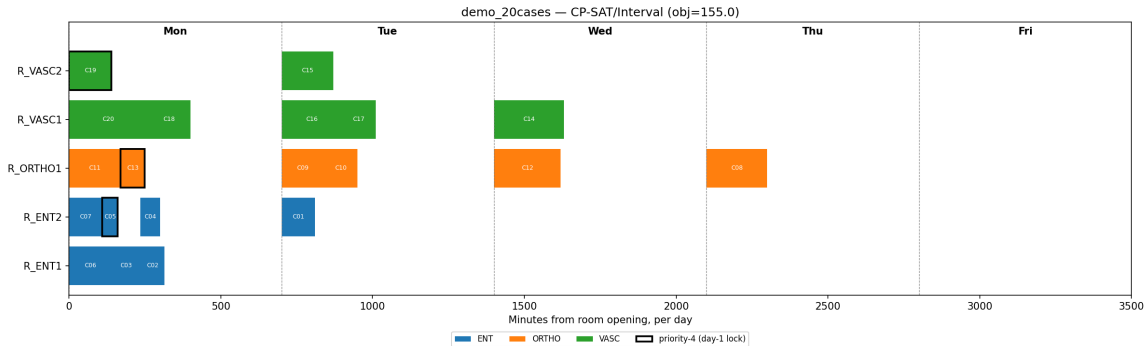
The demo's effect reproduces, larger, at scale: CP-SAT's objective is **6.3% lower** (41,548 vs. 44,346) while scheduling **2 more cases**, even though its own gap (4.30%) looks less converged than CBC's (0.31%).

Reading This Correctly

- **A tighter own-gap is not a better answer.** CBC's 0.31% gap is tight because its feasible region is *smaller* – a day-bucket relaxation that structurally excludes schedules CP-SAT can reach. It's easier to prove there's nothing better than 5 in $\{1, \dots, 5\}$ than in $\{1, \dots, 100\}$; that doesn't make 5 the better answer.
- **Same mechanism at both scales.** The 20-case C-arm story and the 200-case 6.3%/+2-cases result are the same effect: exact temporal/resource reasoning recovers schedules a coarser aggregate constraint forbids outright.
- **This is why CP-SAT is the primary deliverable**, with the MILP kept only as the comparison that makes the claim falsifiable and, here, confirmed.

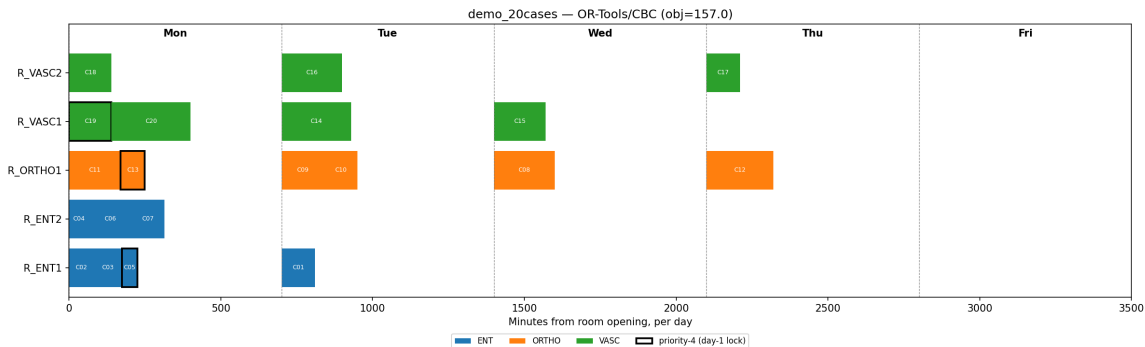
	CP-SAT (primary)	MILP (comparison)
Exact timing/overlap	yes	no
Exact equipment concurrency	yes	no (day-count)
Downstream beds (C11)	yes	not expressible
License	free	free (CBC) / commercial (Gurobi)

Demo Instance – Primary CP-SAT Model



Real start/end clock times. Note Tuesday: two C-arm cases, different rooms, non-overlapping times – the schedule the comparison MILP's equipment cap forbids.

Demo Instance – Comparison MILP (OR-Tools/CBC)



Same cases, no exact clock times (the MILP doesn't model any) – cases within a room-day are laid out back-to-back, arbitrary order.

Medium Instance (200 cases) – Primary CP-SAT Model



Exact per-case

Testing Against Real and Literature Data

`demo_instance()`/`medium_instance()` are synthetic, literature-*structured* (Cardoen et al., 2010). A third instance, `literature_chln_instance()`, is literature-*calibrated*: its waiting-time generator reproduces, by construction, the published 2016 CHLN audit statistics (Marques & Captivo, 2015) – $\sim 16\%$ of cases overdue hospital-wide, ~ 147 days average breach, Neurosurgery specifically ~ 261 days. (Honestly noted: any one finite draw, including the shipped seed, deviates from these targets by sampling noise – itself realistic.)

For testing against real hospital OR logs at full scale, two CC BY-4.0 datasets are a direct structural fit (same weekly horizon, same master-roster concept):

- Akbarzadeh & Maenhout (2023). *Real life data for operating room scheduling problem* – Ghent University Hospital, May 2017. Mendeley Data, 10.17632/n2v49z2vnp.2.
- Akbarzadeh & Maenhout (2023). *RealLife operating room scheduling dataset, 2021-Jan-May* – 20 weekly instances. Mendeley Data, 10.17632/c8d342266x.1.

A loader is the natural next step before a real pilot – intentionally not built here, per this being a small demo, not a production deployment.

Q1 – Passing the Torch

I'd hand a developer four things, not just the math:

- 1 **FORMULATION.md** + `types.py` – the dataclasses *are* the data dictionary; sets/parameters map 1:1 to fields.
- 2 **The solver code itself** – every constraint is labeled (C1...C11) and the matching code carries the same label. Read side by side, no ambiguity about which line implements which formula.
- 3 `test_model.py` **as the acceptance contract** – any reimplementations must pass the same hard-constraint checks on the same demo instance; I'd ask for a test per new constraint before writing the constraint.
- 4 **A short glossary** of the handful of non-obvious domain terms (room roster, ambulatory, priority tiers). Most miscommunication on these projects is vocabulary, not math.

Q2 – A Library of Models

Four layers, solver-agnostic except the bottom one:

- 1 Core data abstractions – typed dataclasses, no solver imports.
- 2 Reusable constraint *patterns* (capacity-sum, NoOverlap, tiered-priority tardiness, eligibility pre-filter) that recur across scheduling problems – nurse rostering and bed allocation need the same shapes, not the same model.
- 3 Problem templates that compose those patterns – this repo's model is one such template.
- 4 A thin solver-adapter layer, one file per backend family (MILP, CP, local search), so a new problem picks a backend without rewriting how constraints are expressed.

The CP-vs-MILP comparison this deck demonstrates is itself a template for that last layer: justify the backend choice from the problem's structure first, verify it empirically on a small instance, *then* commit – rather than defaulting to whichever backend is most familiar.

Extensions and Future Work

Extension	Approach
Stochastic durations	Two-stage stochastic program: first stage selects/places cases, second stage absorbs duration realisations via overtime or case bump
Real-time rescheduling	Large Neighbourhood Search seeded from the current schedule, for same-day disruptions – Hexaly's local-search engine is a natural fit
Nurse/anaesthetist rostering	Extend H to support staff; same NoOverlap/sum pattern as surgeons
Multi-week rolling horizon	Solve weekly; carry forward unscheduled cases with increased priority weight
Day-varying bed capacity	Replace the constant β_p with a per-day Cumulative or reservoir decomposition

- 1 Cardoen, B., Demeulemeester, E., & Beliën, J. (2010). Operating room planning and scheduling: A literature review. *EJOR*, 201(3), 921–932.
- 2 Marques, I., & Captivo, M.E. (2015). *Planeamento de cirurgias eletivas no Centro Hospitalar Lisboa Norte*. MSc thesis, Universidade de Lisboa.
- 3 Denton, B.T., Miller, A.J., Balasubramanian, H.J., & Huschka, T.R. (2010). Optimal allocation of surgery blocks to operating rooms under uncertainty. *Operations Research*, 58(4), 802–816.
- 4 SIGIC – Sistema Integrado de Gestão de Inscritos para Cirurgia, Portaria n.º 45/2008, Diário da República, Portugal.
- 5 Akbarzadeh, B., & Maenhout, B. (2023). Real life data for operating room scheduling problem [Data set]. Mendeley Data, V2. <https://doi.org/10.17632/n2v49z2vnp.2>
- 6 Akbarzadeh, B., & Maenhout, B. (2023). RealLife operating room scheduling dataset, 2021-Jan-May [Data set]. Mendeley Data, V1. <https://doi.org/10.17632/c8d342266x.1>
- 7 Perron, L., & Furnon, V. *CP-SAT: a Constraint Programming Solver* (Google OR-Tools documentation). <https://developers.google.com/optimization/cp>
- 8 Baptiste, P., Le Pape, C., & Nuijten, W. (2001). *Constraint-Based Scheduling*. Kluwer Academic Publishers.
- 9 Vilím, P. (2004). $O(n \log n)$ filtering algorithms for unary resource constraint. *CPAIOR 2004*.
- 10 Schutt, A., Feydy, T., Stuckey, P.J., & Wallace, M.G. (2009). Why cumulative decomposition is not as bad as it sounds. *CP 2009*.

Thank You

Repository: FORMULATION.md · RESULTS.md · README.md

`python main.py` (primary model) `python main.py -benchmark` (comparison)